



المحاضرة 1

تنظيم الحاسوب

Computer Organization

محتويات المحاضرة

1 مكونات نظام الحاسوب

2 لغات البرمجة

3 أنواع المترجمات

4 ما هو ++C؟

5 أساسيات البرمجة بلغة ++C

6 كتابة المخرجات وإضافة التعليقات

1. مكونات نظام الحاسوب

معمارية فون نيومان (Von-Neumann Architecture)

يتكون نظام الحاسوب من المكونات المنطقية التالية:

- **وحدة الإدخال (Input Unit)** – الحصول على البيانات من أجهزة الإدخال (لوحة المفاتيح، الفأرة، الماسح الضوئي، ...).
- **وحدة الإخراج (Output Unit)** – عرض المعلومات على أجهزة الإخراج (الشاشة، الطابعة، ...).
- **وحدة المعالجة المركزية (CPU)** – تنسيق عمل الأقسام الأخرى.
- **وحدة الحساب والمنطق (ALU)** – إجراء العمليات الحسابية والعلاقاتية والمنطقية.
- **وحدة التحكم (Control Unit)** – توليد إشارات التحكم لتنفيذ التعليمات.
- **الذاكرة الرئيسية (Main Memory)** – تخزين قصير المدى للبيانات والبرامج (RAM).
- **الذاكرة الثانوية (Secondary Storage)** – تخزين دائم طويل المدى.

مقارنة: الذاكرة الرئيسية vs. الثانوية

الخاصية	الذاكرة الرئيسية	الذاكرة الثانوية
المدة	مؤقتة، متطايرة	دائمة، غير متطايرة
السرعة	سريعة الوصول	أبطأ
السعة	صغيرة	كبيرة
التكلفة	مرتفعة	منخفضة

2. لغات البرمجة

الجيل الأول - لغة الآلة (Machine Language)

- اللغة الوحيدة التي يفهمها الحاسوب مباشرة.
- تعتمد على الحاسوب (Machine Dependent).
- تتكون من رموز ثنائية للبيانات والتعليمات.

```
0010 0000 0000 0100 0100 0000 0000 0101 0011 0000 0000 0110
```

الجيل الثاني - لغة التجميع (Assembly Language)

- تستخدم اختصارات شبيهة باللغة الإنجليزية.
- تعتمد على الحاسوب (Machine Dependent).
- تحتاج إلى مترجم تجميع (Assembler) لتحويلها إلى لغة الآلة.

```
LOAD BASEPAY
ADD OVERPAY
STORE GROSSPAY
```

الجيل الثالث - لغات عالية المستوى (Compiler Languages)

- شبيهة باللغة الإنجليزية، مستقلة عن الحاسوب (Machine Independent).
- كل تعليمة في اللغة عالية المستوى تُترجم إلى عدة تعليمات في لغة الآلة.
- تحتاج إلى مترجم (Compiler) لتحويلها إلى لغة الآلة.
- أمثلة: FORTRAN, BASIC, Pascal, C, C++, Java, Python.

```
GROSSPAY = BASEPAY + OVERPAY;
```

3. المترجم والمفسر (Compiler vs. Interpreter)

المفسر (Interpreter)

- يترجم البرنامج سطرًا سطرًا .
- إذا حدث خطأ، يتوقف حتى يصحح المبرمج الخطأ.
- أبطأ من المترجم، لكنه أسهل في التصحيح.
- يستخدم مع: BASIC, Python, ...

المترجم (Compiler)

- يترجم البرنامج مرة واحدة كاملة .
- إذا حدثت أخطاء، تظهر بعد الترجمة مباشرة.
- سريع جداً.
- يستخدم مع: C, C++, Java, ...

خطوات إنشاء ملف تنفيذي (exe) في C++

1. إنشاء ملف المصدر (Source Code File) (.cpp)
 2. تجميع الملف باستخدام المترجم → ملف كائن (Object File) (.o)
 3. ربط الملف الكائن مع المكتبات المطلوبة باستخدام الرابط (Linker) → ملف تنفيذي (exe)
- Source (.cpp) → Compiler → Object (.o) → Linker → Executable (.exe)

4. ما هي لغة ++C؟

- ++C هي لغة برمجة شاملة (General-Purpose) و موجهة للكائنات (OOP).
- طوّرها بيارن ستروستروب (Bjarne Stroustrup) عام 1983 كامتداد للغة C.
- تُعتبر لغة متوسطة المستوى – تجمع بين ميزات اللغات عالية ومنخفضة المستوى.
- كان اسمها في البداية "C with Classes" ثم عُيّر إلى ++C.

🔑 خصائص لغة ++C

- لغة حساسة لحالة الأحرف (Case Sensitive) – DATA ≠ Data ≠ data.
- كل تعليمة تنتهي بـ ; (فاصلة منقوطة).
- تحتوي على دالة main() حيث يبدأ تنفيذ الأوامر.
- تستخدم مكتبات (Header Files) مثل `#include <iostream>`.

💡 لماذا ندرس ++C؟

- لغة مهيمنة في بيئات Unix و Windows.
- تركيبها (Syntax) ألهم العديد من اللغات الشهيرة (Java, Python, Ruby).
- قابلة للنقل (Portable) – تعمل على جميع الأجهزة.
- قريبة من الآلة وتعطي إمكانيات غير محدودة.

5. كتابة المخرجات على الشاشة

كائن `cout`

- يُستخدم كائن `cout` لعرض البيانات على الشاشة.
- يُستخدم مع عامل الإدراج `<<`.
- النص المراد عرضه يُكتب بين علامتي تنصيص `" "`.

```
cout << "Hello Students" ;  
cout << "This is your first C++ example" ;
```

مثال كامل

```
#include <iostream>  
using namespace std ;  
  
int main ()  
{  
    cout << "Hello Students" ;  
    cout << "This is your first C++ example" ;  
    return 0 ;  
}
```

المخرجات: Hello StudentsThis is your first C++ example

بدء سطر جديد

- طريقة 1: استخدام `endl`
- طريقة 2: استخدام `\n`

```
cout << "Hello Students" << endl ;  
cout << "This is your first C++ example" << endl ;  
  
cout << "Hello Students \n" ;  
cout << "This is your first C++ example" ;
```

التعليقات

6. إضافة التعليقات والإدخال والإخراج

التعليقات (Comments)

- التعليقات هي وثائق داخل الكود لتذكير المبرمج بوظيفة الأجزاء.
- لا تُرى ولا تُترجم من قبل المترجم.
- أنواع التعليقات في C++:
 - `//` - تعليق سطر واحد.
 - `/* ... */` - تعليق متعدد الأسطر.

```
// This is a single-line comment  
/* This is a  
multi-line comment */
```

مثال كامل مع تعليقات

```
#include <iostream>
using namespace std ;

int main ()
{
    // This is a comment
    cout << "Hello Students" ;
    /* This is a comment too */
    cout << "This is your first C++ example" ;
    return 0 ; // You can add comments any place
}
```

مثال: الإدخال والإخراج (cin / cout)

```
#include <iostream>
using namespace std ;

int main ()
{
    int i ;
    cout << "Please enter an integer value: " ;
    cin >> i ;
    cout << "The value you entered is " << i ;
    cout << " and its double is " << i * 2 << ".\n" ;
    return 0 ;
}
```

المخرجات:

Please enter an integer value: 702
The value you entered is 702 and its double is 1404.

النقاط الأساسية

لغات البرمجة: لغة آلة ← لغة تجميع ← لغات عالية المستوى. 

الحاسوب يتكون من: إدخال، إخراج، CPU، ALU، وحدة تحكم، ذاكرة رئيسية وثنائية. 

C++ لغة حساسة لحالة الأحرف، تنتهي التعليمات بـ ;، وتحتوي على main(). 

المترجم يترجم الكود كاملاً، والمفسر يترجم سطرًا سطرًا. 

التعليقات: // لسطر واحد، /* ... */ لعدة أسطر. 

<<cout لعرض البيانات، >>cin لإدخال البيانات. 

1

Which of the following is NOT a component of the Von-Neumann architecture?

A Memory Unit

B Graphics Processing Unit (GPU)

C Input Unit

D Output Unit

2

What does the ALU (Arithmetic & Logical Unit) do?

A Performs calculations, relational and logical operations

B Generates control signals

C Stores data permanently

D Takes information from the computer to output devices

3

Which of the following is a characteristic of primary memory (RAM)?

A Permanent, non-volatile

B Low cost

C Temporary, volatile

D High capacity

4

Machine language is considered:

A First generation language (Low level)

B High-level language

C Machine independent

D Third generation language

5

What is an assembler used for?

A To execute programs line by line

B To convert assembly language to machine language

C To convert high-level language to machine language

D To debug programs

6

A compiler translates:

A

Assembly language to machine language

B

Only syntax errors

C

The whole program at once into machine language

D

The program line by line

7

Which of the following is a Procedural Language?

A

Java

B

Python

C

C++

D

C

8

What is the difference between an interpreter and a compiler?

A

Interpreter is faster than compiler

B

Compiler translates line by line; interpreter translates the whole program

C

Interpreter translates line by line; compiler translates the whole program

D

There is no difference

9

What is the correct order to create an executable file (.exe) from C++ code?

A

Source code → Interpreter → Object file → Linker → Executable file

B

Source code → Linker → Object file → Compiler → Executable file

C

Source code → Assembler → Object file → Compiler → Executable file

D

Source code → Compiler → Object file → Linker → Executable file

10

Who developed the C++ programming language?

A

Bjarne Stroustrup

B

Guido van Rossum

C

Dennis Ritchie

D

James Gosling

11

What was C++ originally called?

A

C with Classes

B

C++ with Classes

C

Objective C

D

C with Objects

12

C++ is considered a:

A

Machine language

B

Pure object-oriented language

C

Pure procedural language

D

Hybrid language (both procedural and object-oriented)

13

Which header file is needed to use cout in C++?

A

#include <conio.h>

B

#include <stdio.h>

C

#include <iostream>

D

#include <string.h>

14

What does the following statement do?

```
cout << "Hello Students";
```

A

Compiles the program

B

Reads input from the user

C

Displays "Hello Students" on the screen

D

Stores "Hello Students" in memory

15

What is the purpose of "using namespace std;" in a C++ program?

A

To start the main function

B

To declare a new variable

C

To end the program

D

To use standard library functions and objects

16

What does the following statement do?

```
cin >> i;
```

A Multiplies i by 2

B Reads input from the user and stores it in i

C Checks if i is even

D Displays the value of i

17

What is the effect of writing "return 0;" at the end of the main function?

A It starts the program

B It indicates the program executed successfully

C It declares a variable named return

D It prints 0 to the screen

18

Which of the following is an example of an escape sequence to start a new line?

A \r

B \t

C \b

D \n

19

How do you write a single-line comment in C++?

A ' comment '

B // comment

C /* comment */

D # comment

20

What type of error occurs when the program runs but produces incorrect output?

A Syntax error

B Execution error

C Linker error

D Logic error



المحاضرة 2

استخدام المتغيرات لتخزين البيانات

Using Variables to Store Data

محتويات المحاضرة

1 ما هي المتغيرات؟

2 تعريف المتغيرات (Variable Declaration)

3 اختيار اسم المتغير

4 أنواع البيانات (Data Types)

5 إدخال البيانات (cin)

6 الثوابت (Constants)

1. ما هي المتغيرات؟

المتغير (Variable) هو موقع تخزين مؤقت في الذاكرة الرئيسية (RAM) لتخزين البيانات.

- يُستخدم لتخزين البيانات المدخلة من المستخدم أو نتائج العمليات داخل البرنامج.
- يجب تحديد نوع البيانات و الاسم (المعرف).

أسباب استخدام المتغيرات

- لتخزين البيانات التي يدخلها المستخدم.
- لتخزين البيانات أثناء إجراء العمليات عليها.

مثال: حساب المتوسط بدون متغيرات

```
// mean1.cpp
// This program calculates the mean of three numbers.

#include <iostream>
using namespace std ;

int main ()
{
    cout << "First Arithmetic Program \n\n" ;
    cout << ( 12 + 5 + 10 ) / 3 ;
    return 0 ;
} // end main
```

المخرجات:

First Arithmetic Program
9

2. تعريف المتغيرات

ما هو تعريف المتغير؟

- تعريف المتغير يُخبر المترجم بحجز مساحة في الذاكرة لحمل متغير من نوع معين واسم معين.
- يجب تعريف المتغير قبل استخدامه لأول مرة.
- صيغة التعريف: `data_type variable_name`;

خصائص المتغير

- الاسم - لتمييز المتغيرات عن بعضها.
- النوع - يحدد طريقة تخزين البيانات وحجمها في الذاكرة.
- القيمة - البيانات المخزنة في المتغير.
- الموقع في الذاكرة - عنوان في الذاكرة الرئيسية (RAM).

أمثلة على تعريف المتغيرات

```
// تعريف متغير واحد
char letter ;
int x ;
float radius ;

// تعريف أكثر من متغير في سطر واحد
char letter1 , letter2 ;
int w1 , w2 , w3 , w4 ;
float x , y ;
```


3. اختيار اسم المتغير

قواعد تسمية المتغيرات

- يجب أن يكون الاسم ذو معنى لتوثيق البرنامج.
- يمكن استخدام الحروف والأرقام والشرطة السفلية _ .
- ممنوع استخدام المسافات أو الرموز الخاصة.
- يجب أن يبدأ بحرف أو شرطة سفلية _ (لا يبدأ برقم).
- أفضل ألا يزيد عن 31 حرفاً .
- ++C حساسة لحالة الأحرف - Data ≠ data .
- ممنوع استخدام الكلمات المحجوزة (Reserved Words) مثل: int, float, return, const, void, ...

أمثلة صحيحة وخاطئة

❌ خاطئ

1value (يبدأ برقم)
my name (يحتوي على مسافة)
int (كلمة محجوزة)
\$total (رمز خاص)

✅ صحيح

age
student_name
counter_
value1

4. أنواع البيانات

الأنواع المضمنة (Built-in Types)

النوع	الوصف	المساحة
int	أعداد صحيحة (بدون فاصلة عشرية)	4 بايت
float	أعداد حقيقية (بفاصلة عشرية)	4 بايت
double	أعداد حقيقية (دقة مضاعفة)	8 بايت
char	حرف واحد	1 بايت
bool	قيمة منطقية (صحيح/خطأ)	1 بايت
string	سلسلة من الأحرف	متغير

نطاق الأعداد الصحيحة

النوع	الحجم	النطاق
short	2 بايت	-32,768 إلى 32,767
int	4 بايت	-2,147,483,648 إلى 2,147,483,647
long long	8 بايت	- $10^{18} \times 9.22$ إلى $10^{18} \times 9.22$

تابع: أنواع البيانات

نوع char

- يستخدم لتخزين حرف واحد .
- يُكتب الحرف بين علامتي تنصيص مفردة ' ' .
- مثال: char grade = 'A';
- يخزن قيمة ASCII للحرف (0-255).

```
char c = 64 ;
cout << c ; // يطبع '@'
c = c + 1 ;
cout << c ; // يطبع 'A'
```

التحويل بين char و int

```
char c = 'A' ;
cout << c << " is the same as " << int ( c ) ;
// يطبع: A is the same as 65
```

جدول ASCII

65	66	67	68	69	70	71	72
A	B	C	D	E	F	G	H
73	74	75	76	77	78	79	80
I	J	K	L	M	N	O	P
81	82	83	84	85	86	87	88
Q	R	S	T	U	V	W	X
89	90	97	98	99	100	101	102
Y	Z	a	b	c	d	e	f

كل حرف له قيمة عددية تسمى ASCII code.

5. إدخال البيانات (cin)

كائن cin

- يستخدم **cin** لقراءة البيانات من لوحة المفاتيح.
- موجود في ملف **iostream**.
- صيغة قراءة قيمة واحدة: **cin >> variable**;

```
int length ;  
cin >> length ;
```

قراءة قيم متعددة

```
int length , width ;  
cin >> length >> width ;
```

يمكن للمستخدم إدخال القيم مفصولة بمسافات أو أسطر جديدة.

مثال: حساب المتوسط باستخدام المتغيرات

```
// mean5.cpp
// This program calculates the mean of ANY three numbers.

#include <iostream>
using namespace std ;

int main ()
{
    float num1 , num2 , num3 , mean ;

    cout << "Enter first number: " ;
    cin >> num1 ;
    cout << "Enter second number: " ;
    cin >> num2 ;
    cout << "Enter third number: " ;
    cin >> num3 ;

    mean = ( num1 + num2 + num3 ) / 3.0 ;

    cout << "The average of " << num1 << " and " << num2 << " and " <<
    num3 ;
    cout << " is equal to = " << mean << endl ;
    return 0 ;
} // end main
```

المخرجات:

```
Enter first number: 2
Enter second number: 5
Enter third number: 6
The average of 2 and 5 and 6 is equal to = 4.33333
```

6. الثوابت (Constants)

ما هي الثوابت؟

- الثوابت هي قيم **لا تتغير** طوال تنفيذ البرنامج.
- تُستخدم لإعطاء أسماء ذات معنى لقيم ثابتة.
- من الأفضل استخدام **أحرف كبيرة** لأسماء الثوابت.

```
const float PI = 3.14159 ;  
const float METERS_TO_YARDS = 1.196 ;
```

صيغة تعريف الثابت

```
const datatype CONSTANT_NAME = VALUE;
```

النقاط الأساسية

تعريف المتغير: `data_type`
;`variable_name`

المتغيرات تخزن البيانات مؤقتاً في الذاكرة الرئيسية (RAM).

`cin` للقراءة، `cout` للكتابة.

أنواع البيانات: `int`, `float`, `double`, `char`, `bool`, `string`.

⚠ انتبه لقسمة الأعداد الصحيحة: $3/2 = 1$,
بينما $3.0/2 = 1.5$.

🔒 الثوابت (`const`) تحفظ قيمة لا تتغير.

⚡ خلاصة سريعة

- المتغيرات تخزن البيانات مؤقتاً في الذاكرة الرئيسية.
- يجب تحديد نوع البيانات والاسم عند تعريف المتغير.
- أسماء المتغيرات يجب أن تكون ذات معنى وتتبع قواعد التسمية.
- استخدم `const` للثوابت التي لا تتغير أثناء تنفيذ البرنامج.
- انتبه لقسمة الأعداد الصحيحة – استخدم `3.0` بدلاً من `3` للحصول على نتيجة صحيحة.

1

A variable in C++ is:

A

A temporary storage location in main memory (RAM)

B

A permanent storage location in secondary memory

C

A programming language

D

A type of compiler

2

Which of the following is NOT a reason to use variables?

A

To temporarily store data in RAM

B

To permanently store data on hard drive

C

To hold data inputted from user

D

To hold data for processing through operations

3

What does a variable declaration do?

A

Informs the compiler to set aside space in memory for the variable

B

Assigns a value to the variable

C

Deletes the variable from memory

D

Prints the variable value to screen

4

Which of the following is a valid variable name in C++?

A

my-var

B

int

C

my_var

D

2ndNumber

5

What is the size of an int data type on most systems (64-bit GCC)?

A

2 bytes

B

1 byte

C

8 bytes

D

4 bytes

6

Which data type is used to store a single character in C++?

A string

B float

C char

D int

7

What is the range of an unsigned short int on most systems?

A 0 to 65,535

B -128 to +127

C 0 to 255

D -32,768 to +32,767

8

Which of the following is NOT a predefined data type in C++?

A string

B int

C float

D char

9

What is the ASCII code of the character 'A'?

A 64

B 67

C 66

D 65

10

What is the output of the following code?

```
char c = 64; cout << c;
```

A Error

B 64

C A

D @

11

How do you declare a constant in C++?

A `const float PI = 3.14159;`

B `constant float PI = 3.14159;`

C `float PI = 3.14159;`

D `#define PI 3.14159`

12

What does the `cin` object do?

A Performs arithmetic operations

B Reads input from the keyboard

C Declares variables

D Displays output on the screen

13

What is the output of the following code?

```
int a = 12, b = 5, c = 10;
```

```
float mean = (a + b + c) / 3;
```

```
cout << mean;
```

A 9.66667

B Error

C 9

D 9.0

14

How can you read multiple values using `cin` in one line?

A Both A and B are correct

B `cin >> a >> b;`

C None of the above

D `cin >> a; cin >> b;`

15

Which of the following is a user-defined data type in C++?

A `float`

B `char`

C `int`

D `struct`

16

What is the size of a char data type in C++?

A 2 bytes

B 1 byte

C 4 bytes

D 8 bytes

17

What is the result of integer division?

int result = 10 / 3;

A 3.0

B 3.33333

C 3

D 4

18

What is the size of a double data type on most systems?

A 8 bytes

B 2 bytes

C 16 bytes

D 4 bytes

19

What does the following code do?

```
char name[12];
```

A Declares a string variable that can hold 12 characters

B Declares a float variable

C Declares an integer array of size 12

D Declares a string variable that can hold 11 characters (plus null terminator)

20

Which of the following is synonymous with "short int"?

A unsigned

B int

C short

D long



المحاضرة 3

العمليات الحسابية

Arithmetic Operators

محتويات المحاضرة

1 العمليات الحسابية (Arithmetic Operators)

2 ترتيب العمليات (Order of Precedence)

3 الإسناد (Assignment)

4 عمليات الإسناد المختصرة (Augmented Assignment)

5 الزيادة والإنقاص (Increment/Decrement)

6 تحويل الأنواع (Type Casting)

7 دوال المكتبة الرياضية (Math Library)

1. العمليات الحسابية

المشغلات الحسابية (Arithmetic Operators)

العملية	المشغل	مثال
الجمع	+	$a + b$
الطرح	-	$a - b$
الضرب	*	$a * b$
القسمة	/	a / b
الباقى	%	$a \% b$

مثال على القسمة والباقي

```
cout << 5 / 3 ; // 1 (قسمة صحيحة) يطبع
cout << 5.0 / 2 ; // 2.5 يطبع
cout << 5 % 2 ; // 1 (باقي القسمة) يطبع
```

2. ترتيب العمليات

أولوية المشغلات (Order of Precedence)

الأولوية	المشغل	الوصف
1	()	الأقواس
2	% / *	الضرب والقسمة والباقي
3	- +	الجمع والطرح

إذا احتوت العبارة على مشغلات متماثلة، تُنفذ من اليسار إلى اليمين .

مثال: ترتيب التنفيذ

$$3 + 4 * 4 + 5 * (4 + 3) - 1$$

(1) داخل الأقواس أولاً $\rightarrow 3 + 4 * 4 + 5 * 7 - 1$

(2) الضرب $\rightarrow 3 + 16 + 35 - 1$

(3) الضرب $\rightarrow 3 + 16 + 35 - 1$

(4) الجمع $\rightarrow 3 + 54 - 1$

(5) الجمع $\rightarrow 57 - 1$

(6) الطرح $\rightarrow 56$

جرب بنفسك

ما ناتج العبارة التالية؟

$$z = p * r \% q + w / x - y;$$

ترتيب التنفيذ: $1 * -2 \% -3 / -4 + -5 -$

3. الإسناد

صيغة الإسناد (Assignment)

$$; \text{variable} = \text{expression}$$

- تُستخدم الإسناد لتخزين قيمة في متغير.
- يُحسب التعبير أولاً، ثم تُخزن النتيجة في المتغير.

```
int c = 5 ;
c = c + 3 ; // تصبح c 8
```

الإسناد المختصر

4. عمليات الإسناد المختصرة

المشغلات المختصرة (Augmented Assignment)

المشغل	يساوي
=+	$a = a + b$
=-	$a = a - b$
=*	$a = a * b$
=/	$a = a / b$
=%	$a = a \% b$

```
int c = 5 ;
c += 3 ; // نفس c = c + 3
```

5. الزيادة والإنقاص

مشغلات ++ و --

المشغل	الاسم	الوصف
var++	ما قبل الزيادة	يزيد var بمقدار 1، ثم يُقَيَّم إلى القيمة الجديدة
++var	ما بعد الزيادة	يُقَيَّم إلى القيمة الأصلية، ثم يزيد var بمقدار 1
var--	ما قبل الإنقاص	ينقص var بمقدار 1، ثم يُقَيَّم إلى القيمة الجديدة
--var	ما بعد الإنقاص	يُقَيَّم إلى القيمة الأصلية، ثم ينقص var بمقدار 1

مثال توضيحي

```
int c = 5 ;
cout << c ++ ; // إلى 6 c يطبع 5، ثم يزيد
cout << c ; // 6 يطبع

c = 5 ;
cout << ++ c ; // إلى 6، ثم يطبع 6 c يزيد
cout << c ; // 6 يطبع
```


6. تحويل الأنواع (Type Casting)

التحويل الضمني (Implicit Casting)

- يتم تلقائياً بواسطة المترجم.
- قد ينتج عنه فقدان في البيانات.

```
int i = 6.5 ; // i = 6 (0.5 تجاهل)
double d = 3 ; // d = 3.0
```

التحويل الصريح (Explicit Casting)

- يقوم به المبرمج باستخدام static_cast .
- يفضل استخدامه لتجنب التحذيرات من المترجم.

```
double d = 4.5 ;
int i = static_cast < int > ( d ) ; // i = 4
```

ملاحظة مهمة

- التحويل لا يغير قيمة المتغير الأصلي.
- static_cast<int>(d) لا يغير قيمة d.

7. دوال المكتبة الرياضية

دوال cmath

- يجب تضمين `<cmath>` باستخدام `<include <cmath>`

الدالة	الوصف
<code>abs(a)</code>	القيمة المطلقة لـ a
<code>pow(a, b)</code>	a مرفوعة للقوة b
<code>sqrt(a)</code>	الجذر التربيعي لـ a
<code>sin(a), cos(a), tan(a)</code>	الدوال المثلثية (a بالراديان)
<code>log(a)</code>	اللوغاريتم الطبيعي
<code>log10(a)</code>	اللوغاريتم للأساس 10
<code>exp(a)</code>	e مرفوعة للقوة a

تابع دوال المكتبة الرياضية

مثال: استخدام pow()

```
#include <iostream>
#include <cmath>
using namespace std ;

int main ()
{
    cout << pow ( 2.0 , 3 ) << endl ; // 8
    cout << pow ( 4.0 , 0.5 ) << endl ; // 2
    cout << pow ( 2.5 , 2 ) << endl ; // 6.25
    return 0 ;
}
```

الأخطاء الشائعة

✗ متغيرات غير معرفة أو غير مهيأة

```
double interestRate = 0.05 ;
double interest = interestrates * 45 ; // خطأ إملائي
```

✗ قسمة صحيحة غير مقصودة

```
int number1 = 1 ;
int number2 = 2 ;
double average = ( number1 + number2 ) / 2 ; // الناتج 1.0 وليس 1.5
```

✓ الحل: استخدم / 2.0

✗ نسيان تضمين ملفات الرأس

```
// نسيان #include <cmath>
cout << pow ( 2.0 , 3 ) ; // خطأ
```

✗ تجاوز سعة العدد الصحيح (Integer Overflow)

```
short value = 32767 + 1 ; // يصبح -32768
```

النقاط الأساسية

ترتيب الأولوية: () ثم * / % ثم + - 

المشغلات الحسابية: +, -, *, /, % 

++ و -- للزيادة والإنقاص بمقدار 1 

الإسناد المختصر: +=, -=, *=, /=, %= 

دوال: pow(), sqrt(), abs(), sin(), cos(), ... 

تحويل الأنواع: static_cast<type>(value) 

خلاصة سريعة

- ++C لا يدعم عملية الأس مباشرة، استخدم pow () من cmath .
- قسمة عددين صحيحين تعطي ناتجاً صحيحاً (يُحذف الكسر).
- استخدم static_cast للتحويل الصريح بين الأنواع.
- انتبه لتجاوز سعة الأعداد الصحيحة (تجاوز النطاق المسموح).

1

Which of the following is NOT a built-in arithmetic operator in C++?

A **

B %

C +

D -

2

What is the result of $5 / 3$ in C++ when both operands are integers?

A 1.5

B 2

C 1.66667

D 1

3

What is the result of $5 \% 2$ in C++?

A 2.5

B 1

C 2

D 0

4

What is the order of precedence for arithmetic operators in C++?

A $+- \rightarrow () \rightarrow * / \%$

B $() \rightarrow * / \% \rightarrow +-$

C $* / \% \rightarrow +- \rightarrow ()$

D $+- \rightarrow * / \% \rightarrow ()$

5

Evaluate the expression: $3 + 4 * 4 + 5 * (4 + 3) - 1$

A 52

B 55

C 54

D 53

6

What does the augmented assignment operator += do?

A `c = c / 3`

B `c = c + 3`

C `c = c - 3`

D `c = c * 3`

7

What is the output of the following code?

```
int c = 5; cout << c++ << endl; cout << c << endl;
```

A 6 then 5

B 5 then 6

C 5 then 5

D 6 then 6

8

What is the output of the following code?

```
int c = 5; cout << ++c << endl; cout << c << endl;
```

A 5 then 6

B 6 then 5

C 5 then 5

D 6 then 6

9

What is implicit casting?

A Converting a variable to a different variable name

B Changing the value of a constant

C Manual conversion using `static_cast`

D Automatic conversion of one data type to another

10

What is the result of the following code?

```
double d = 4.5; int i = static_cast (d);
```

A `i = 5, d = 4.5`

B `i = 4, d = 4`

C `i = 5, d = 5`

D `i = 4, d = 4.5`

11

What header file is needed to use the `pow()` function?

A `#include <cmath>`

B `#include <iostream>`

C `#include <stdio.h>`

D `#include <math.h>`

12

What is the result of `pow(2.0, 3)` in C++?

A 10

B 6

C 8

D 9

13

What is the result of `sqrt(16)` in C++?

A 6

B 5

C 3

D 4

14

What is the output of the following code?

```
int number1 = 1; int number2 = 2;  
double average = (number1 + number2) / 2;  
cout << average;
```

A 2

B 1

C 1.5

D 1.0

15

What is integer overflow?

A When a number exceeds the maximum value that can be stored in a data type

B When a variable is used before initialization

C When a variable is not declared

D When a number is divided by zero

16

What is the result of $5.0 / 2$ in C++?

A 2.0

B 2

C 2.5

D 3

17

What does the modulus operator (%) do?

A Adds two numbers

B Divides two numbers and returns the quotient

C Multiplies two numbers

D Divides two numbers and returns the remainder

18

What is the value of `abs(-5)` in C++?

A -4

B 5

C 0

D -5

19

What is the correct way to calculate $5x^2$ using the `pow()` function?

A `pow(5 * x, 2)`

B `5 * pow(x, 2)`

C `5 * x * x`

D `5 * pow(2, x)`

20

What is the result of `pow(4.0, 0.5)` in C++?

A 1

B 8

C 2

D 4



المحاضرة 4

التحكم في التنفيذ - الاختيار

Control Constructs - Selection

محتويات المحاضرة

1 مقدمة في هياكل التحكم

2 جملة if

3 جملة if/else

4 جملة if/else if

5 الجمل المتداخلة (Nested if)

6 العوامل المنطقية

7 العامل الشرطي (?:)

8 جملة switch

9 توليد الأعداد العشوائية

1. مقدمة في هياكل التحكم

- حتى الآن، كنا نكتب برامج باستخدام **التسلسل البسيط** (التنفيذ بالترتيب).
- في كثير من الحالات، نحتاج إلى **اتخاذ قرار** بناءً على قيمة متغير أو تعبير.
- جملة الشرط تسمح بتنفيذ جمل أخرى فقط تحت شروط معينة.

هياكل الاختيار في ++C

- `if`
- `if/else`
- `if/else if/else`
- `nested if` (جمل متداخلة)
- `switch/case`

2. جملة if

الصيغة العامة

```
if ( condition )
statement ;
```

- تُنفذ الجملة فقط إذا كان الشرط صحيحاً (true) .
- الشرط هو تعبير منطقي (Boolean expression) .

مثال

```
if ( grade >= 60 )
cout << "Passed" ;
```

مثال آخر

```
int score1 , score2 , score3 ;
float average ;
cin >> score1 >> score2 >> score3 ;
average = ( score1 + score2 + score3 ) / 3.0 ;

if ( average > 95 )
cout << "Congratulations! That's a high score!\n" ;
// average > 95 تُنفذ فقط إذا كان
cout << "after the if statement" ;
```

تنفيذ أكثر من جملة

```
if ( x < 20 )
{
    y = 30 ;
    z = 100 ;
    cout << x << "\n" << y << "\n" << z ;
}
```

استخدم الأقواس المتعرجة { } لتنفيذ أكثر من جملة.

جملة IF/ELSE

3. جملة if/else

الصيغة العامة

```
if ( condition )
    statement1 ;
else
    statement2 ;
```

- تُنفذ statement1 إذا كان الشرط صحيحاً.
- تُنفذ statement2 إذا كان الشرط خاطئاً.

مثال: التحقق من النجاح والرسوب

```
if ( grade >= 60 )
    cout << "Passed" ;
else
    cout << "Failed" ;
```

تابع جملة if/else

مثال: التحقق من الزوجية والفردية

```
#include <iostream>
using namespace std ;

int main ()
{
    int number ;
    cout << "Enter an integer: " ;
    cin >> number ;

    if ( number % 2 == 0 )
        cout << number << " is even.\n" ;
    else
        cout << number << " is odd.\n" ;
    return 0 ;
}
```

4. جملة if/else if

الصيغة العامة

```
if ( condition1 )
statement1 ;
else if ( condition2 )
statement2 ;
else if ( condition3 )
statement3 ;
else
statement4 ;
```

- تُنفذ الشروط بالترتيب حتى يتم العثور على شرط صحيح.
- إذا لم يكن أي شرط صحيحاً، تُنفذ جملة else (إن وُجدت).

مثال: تحويل الدرجة إلى حرف

```
if ( score > 100 )
cout << "Error: score is out of range." ;
else if ( score >= 90 )
cout << 'A' ;
else if ( score >= 80 )
cout << 'B' ;
else if ( score >= 70 )
cout << 'C' ;
else if ( score >= 60 )
cout << 'D' ;
else if ( score >= 0 )
cout << 'F' ;
else
cout << "Error: score is out of range." ;
```

5. الجملة المتداخلة (Nested if)

ما هي الجملة المتداخلة؟

- جملة if داخل جملة if أخرى.
- تُستخدم لاتخاذ قرارات معقدة.

مثال

```
if ( x > y ) // if خارجي
{
    if ( x == 0 ) // if داخلي
    // أي كود
else
    // أي كود
}
else
    // أي كود
```

⚠ ملاحظة مهمة

- جملة else ترتبط بأقرب جملة if غير مقترنة.

```
int i = 1 , j = 2 , k = 3 ;
if ( i > j )
{
    if ( i > k )
    cout << "A" ;
}
else
    cout << "B" ; // تطبع B
```


6. العوامل المنطقية

العوامل المنطقية (Logical Operators)

العامل	المعنى	التأثير
&&	AND	كلا الشرطين يجب أن يكونا صحيحين
	OR	أحد الشرطين أو كلاهما يجب أن يكون صحيحاً
!	NOT	يعكس قيمة الشرط

```
if ( ( x > 0 ) && ( a == b ) )
// x > 0 و a == b يُنفذ إذا كان
```

العامل الشرطي

7. العامل الشرطي (?:)

الصيغة العامة

```
condition ? expression1 : expression2;
```

- إذا كان الشرط صحيحاً → يُنفذ expression1 .
- إذا كان الشرط خاطئاً → يُنفذ expression2 .
- هذا هو العامل الثلاثي الوحيد في ++C.

مثال

```
cout << ( grade >= 60 ? "Passed" : "Failed" ) ;
```

نفس if/else لكن بشكل أكثر إحكاماً.

8. جملة switch

الصيغة العامة

```
switch ( expression )
{
case value1 :
// statements
break ;
case value2 :
// statements
break ;
default :
// statements
break ;
}
```

- تختبر قيمة تعبير صحيح وتنتقل إلى الحالة المنطابقة.
- break يمنع تسلسل التنفيذ (fall through).
- default اختياري، يُنفذ إذا لم تتطابق أي حالة.

مثال 1: اختيار رقم

```
switch ( i )
{
case 1 :
cout << "one is entered" << endl ;
break ;
case 2 :
cout << "two is entered" << endl ;
break ;
case 3 :
cout << "three is entered" << endl ;
break ;
default :
cout << "number other than 1, 2 or 3 is entered" << endl ;
break ;
}
```

مثال 2: تحويل الدرجة إلى حرف (باستخدام switch)

```
switch ( score / 10 )
{
case 10 :
case 9 : cout << 'A' << endl ; break ;
case 8 : cout << 'B' << endl ; break ;
case 7 : cout << 'C' << endl ; break ;
case 6 : cout << 'D' << endl ; break ;
case 5 :
case 4 :
case 3 :
case 2 :
case 1 :
case 0 : cout << 'F' << endl ; break ;
default : cout << "Error: score is out of range.\n" ;
}
```

9. توليد الأعداد العشوائية

دالة rand()

- تُستخدم دالة rand () للحصول على عدد صحيح عشوائي.
- تُرجع عدداً بين 0 و RAND_MAX (32767 في Visual C++).
- للحصول على عدد عشوائي بين 0 و 9: rand() % 10

```
#include <iostream>
#include <cstdlib> // rand() تحتاجها
#include <ctime> // time() تحتاجها

srand ( time ( 0 ) ) ; // لبدء تسلسل عشوائي مختلف في كل مرة
int num = rand ( ) % 100 ; // عدد عشوائي بين 0 و 99
```

تابع توليد الأعداد العشوائية

مثال: برنامج تعليمي للطرح

```
// توليد رقمين عشوائيين من 0 إلى 9
int number1 = rand () % 10 ;
int number2 = rand () % 10 ;

// التأكد من أن number1 >= number2
if ( number1 < number2 )
{
    int temp = number1 ;
    number1 = number2 ;
    number2 = temp ;
}

// سؤال الطالب
cout << "What is " << number1 << " - " << number2 << " ? " ;
int answer ;
cin >> answer ;

// التحقق من الإجابة
if ( number1 - number2 == answer )
    cout << "You are correct!" ;
else
    cout << "Your answer is wrong.\n"
    << number1 << " - " << number2
    << " should be " << ( number1 - number2 ) << endl ;
```

النقاط الأساسية

if/else: تنفيذ جملة إذا كان الشرط صحيحاً،
وأخرى إذا كان خاطئاً. 

if: تنفيذ جملة إذا كان الشرط صحيحاً. 

العوامل المنطقية: && (AND)، || (OR)، ! (NOT). 

if/else if: اختبار شروط متعددة بالترتيب. 

switch: اختيار مسار بناءً على قيمة عدد صحيح. 

? العامل الشرطي: اختصار لـ if/else.

خلاصة سريعة

- استخدم { } عند تنفيذ أكثر من جملة.
- لا تنس break في switch لتجنب تسلسل التنفيذ.
- rand () تولد أعداداً عشوائية، و srand () تبدأ تسلسلاً مختلفاً.
- انتبه لاستخدام == (مقارنة) وليس = (إسناد) في الشروط.

1

What is the default control construct in C++ and most procedural languages?

A Iteration

B Recursion

C Simple sequence

D Selection

2

Which of the following is NOT a selection construct in C++?

A if/else

B switch/case

C if

D while

3

What is a boolean expression?

A An expression that declares a variable

B An expression that performs arithmetic operations

C An expression that represents a condition that is either true or false

D An expression that returns a numeric value

4

What is the output of the following code if average = 90?

```
if (average > 95) cout << "Congratulations!"; cout << "after the if statement";
```

A Congratulations!after the if statement

B after the if statement

C Congratulations!

D No output

5

Which relational operator is used to check for equality in C++?

A ==

B !=

C =

D ===

6

What does the logical operator && (AND) do?

A Always returns true

B Reverses the truth of an expression

C One or both expressions must be true for the overall expression to be true

D Both expressions must be true for the overall expression to be true

7

What is the output if grade = 75?

if (grade >= 60) cout << "Passed"; else cout << "Failed";

A Error

B Failed

C No output

D Passed

8

What is the output if number = 5?

if (number % 2 == 0) cout << "even"; else cout << "odd";

A odd

B No output

C even

D Error

9

What is the output if score = 85?

if (score >= 90) cout << 'A'; else if (score >= 80) cout << 'B'; else if (score >= 70) cout << 'C'; else cout << 'F';

A B

B F

C C

D A

10

What does the conditional operator (ternary operator) do?

A It works on two operands

B It works on four operands

C It works on three operands and is a compact if/else

D It works on one operand

11

What is the output of the following code?

```
int i = 1, j = 2, k = 3;
```

```
if (i > j) { if (i > k) cout << "A"; } else cout << "B";
```

A A

B No output

C Error

D B

12

What is the output of the following code?

```
int count = 0;
```

```
if (count = 0) cout << "count is zero"; else cout << "count is not zero";
```

A Error

B count is zero

C count is not zero

D No output

13

What is the syntax for the switch statement in C++?

A switch expression { case value: statements; break; default: statements; }

B switch (expression) { case value: statements; break; default: statements; }

C switch (expression) { case (value) { statements } default { statements } }

D switch (expression) { case value { statements } default { statements } }

14

What is the purpose of the "break" statement in a switch?

A To exit the program

B To start a new case

C To declare a variable

D To prevent fall-through to the next case

15

What is the output if i = 2?

```
switch (i) { case 1: cout << "one"; break; case 2: cout << "two"; break; case 3: cout << "three"; break; default: cout << "other"; }
```

A two

B three

C one

D other

16

What does the rand() function return?

A A random floating-point number

B A random integer between 0 and RAND_MAX

C A random integer between 1 and 10

D A random character

17

How can you generate a random number between 0 and 9 in C++?

A rand() * 10

B rand() % 10

C rand() / 10

D rand() % 9

18

What is the output if score = 50?

```
switch (score/10) { case 10: case 9: cout << 'A'; break; case 8: cout << 'B'; break; case 7: cout << 'C'; break; case 6: cout << 'D'; break; default: cout << 'F'; }
```

A C

B D

C F

D Error

19

What is the output if score = 95?

```
switch (score/10) { case 10: case 9: cout << 'A'; break; case 8: cout << 'B'; break; default: cout << 'F'; }
```

A F

B Error

C A

D B

20

What is the output of the following code?

```
cout << (grade >= 60 ? "Passed" : "Failed"); if grade = 75
```

A No output

B Error

C Passed

D Failed



المحاضرة 5

التحكم في التنفيذ - التكرار

Control Constructs - Iteration

محتويات المحاضرة

1 مقدمة في حلقات التكرار

2 حلقة while

3 حلقة do-while

4 حلقة for

5 جمل continue و break

6 أمثلة تطبيقية

1. مقدمة في حلقات التكرار

لنفترض أنك بحاجة لطباعة جملة "Welcome to C++!" مئة مرة.
سيكون من الممّل كتابة `cout` مئة مرّة!

أنواع حلقات التكرار في C++

- `while` – حلقة تختبر الشرط **قبل** التنفيذ (**pre-test**) .
- `do-while` – حلقة تختبر الشرط **بعد** التنفيذ (**post-test**) .
- `for` – حلقة عدّاد (**counting loop**) .

2. حلقه while

الصيغة العامة

```
while ( condition )
{
    // الجمل التي سيتم تكرارها
}
```

- تُنفذ طالما أن الشرط **صحيح (true)**.
- تُختبر القيمة **قبل** كل تكرار (pre-test).
- إذا كان الشرط خاطئاً من البداية، **لا تُنفذ** نهائياً.

مثال: قراءة أعداد حتى إدخال 99

```
int number = 0 ;
cout << "Enter an integer number.\n" ;
cout << "Enter 99 when you want to quit: " ;

while ( number != 99 )
{
    cin >> number ;
}
cout << "Done.\n" ;
```

⚠ حلقه لا نهائية (Infinite Loop)

- إذا لم يكن للحلقه طريقة للتوقف، تُصبح **حلقه لا نهائية**.

```
int test = 0 ;
while ( test < 10 )
    cout << "Hello\n" ; // test لا نهائية → حلقه لا نهائية
```

تابع: حلقة while مع عداد

العداد (Counter)

- متغير يزداد أو ينقص في كل تكرار.
- يُستخدم للتحكم في عدد مرات التكرار.

مثال: طباعة الأعداد من 1 إلى 10

```
int num = 1 ; // تهيئة العداد
while ( num <= 10 ) // اختبار الشرط
{
    cout << "The number now is " << num << endl ;
    num ++ ; // تحديث العداد
}
```

مثال: طباعة الأعداد ومربعاتها

```
int num = 1 ;
cout << "Number\tNumber Squared\n" ;
cout << "-----\n" ;
while ( num <= 10 )
{
    cout << num << "\t\t" << ( num * num ) << endl ;
    num ++ ;
}
```

```
Number Number Squared
-----
1 1
2 4
3 9
... ...
10 100
```

3. حلقة do-while

الصيغة العامة

```
do
{
    // الجمل التي سيتم تكرارها
} while ( condition ) ;
```

- تُنفذ مرة واحدة على الأقل .
- تُختبر القيمة **بعد** كل تكرار (post-test).
- تنتهي بـ ; (فاصلة منقوطة).

مقارنة بين while و do-while

do-while

```
int x = 1 ;
do
cout << x ;
while ( x < 0 ) ;
```

تُنفذ مرة واحدة (1 مرة).

while

```
int x = 1 ;
while ( x < 0 )
cout << x ;
```

لا تُنفذ أي مرة (0 مرات).

مثال: نفس المثال مع do-while

```
int num = 1 ;
do
{
    cout << "The number now is " << num << endl ;
    num ++ ;
} while ( num <= 10 ) ;
```

4. حلقة for

الصيغة العامة

```
for ( init ; condition ; update )
{
    // الجمل التي سيتم تكرارها
}
```

- **init** – تُنفذ **مرة واحدة** في بداية الحلقة (تهيئة العداد).
- **condition** – تُختبر **قبل** كل تكرار (pre-test).
- **update** – تُنفذ في **نهاية** كل تكرار (تحديث العداد).

مثال: طباعة الأعداد من 1 إلى 10

```
for ( int num = 1 ; num <= 10 ; num ++ )
{
    cout << num << endl ;
}
```

مثال: طباعة الأعداد الفردية من 1 إلى 20

```
for ( int count = 1 ; count <= 20 ; count += 2 )
{
    cout << count << endl ;
}
```


مثال: حساب مجموع الأعداد من 1 إلى 20

```
int sum = 0 ;  
for ( int count = 1 ; count <= 20 ; count ++ )  
{  
    sum += count ;  
}  
cout << "The sum is " << sum << endl ;
```

5. جمل `continue` و `break`

جملة `break`

- تُنهي الحلقة فوراً.
- ينتقل التنفيذ إلى أول جملة بعد الحلقة.

```
for ( int x = 1 ; x <= 10 ; x ++ )
{
    if ( x == 5 )
        break ; // توقف عند x = 5
    cout << x << " " ;
}
// 1 2 3 4
```

جملة `continue`

- تتخطى بقية الجمل في التكرار الحالي.
- تنتقل إلى التكرار التالي.

```
for ( int x = 1 ; x <= 10 ; x ++ )
{
    if ( x == 5 )
        continue ; // يتخطى طباعة 5
    cout << x << " " ;
}
// 1 2 3 4 6 7 8 9 10
```

النقاط الأساسية

do-while  : تختبر الشرط بعد التكرار (post-test) وتنفذ مرة واحدة على الأقل.

while  : تختبر الشرط قبل التكرار (pre-test).

break  : تخرج من الحلقة فوراً.

for  : حلقة عدّاد، مثالية للتكرار لعدد معروف من المرات.

⚠ تجنب الحلقات اللانهائية بتحديث العداد بشكل صحيح.

continue  : تتخطى بقية التكرار وتنتقل للتالي.

خلاصة سريعة

- استخدم while عندما لا تعرف عدد التكرارات مسبقاً.
- استخدم do-while عندما تريد تنفيذ الحلقة مرة واحدة على الأقل.
- استخدم for عندما تعرف عدد التكرارات مسبقاً (عداد).
- تأكد من تحديث العداد لتجنب الحلقات اللانهائية.

1

What type of loop is the while loop?

A

Infinite loop

B

Post-test loop

C

Counting loop

D

Pre-test loop

2

How many times will the following while loop execute?

```
int x = 1; while (x < 0) { cout << x; x++; }
```

A

1 time

B

10 times

C

Infinite times

D

0 times

3

What type of loop is the do-while loop?

A

Pre-test loop

B

Post-test loop

C

Infinite loop

D

Counting loop

4

How many times will the following do-while loop execute?

```
int x = 1; do { cout << x; x++; } while (x < 0);
```

A

10 times

B

0 times

C

1 time

D

Infinite times

5

What is a counter variable in a loop?

A

A variable that is regularly incremented or decremented each iteration

B

A variable that stores the average

C

A variable that ends the program

D

A variable that stores the sum of all values

6

What is the output of the following code?

```
int num = 1; while (num <= 3) { cout << num; num++; }
```

A 123

B 1234

C 12

D No output

7

What is an infinite loop?

A A loop that never stops executing

B A loop that has no body

C A loop that executes once

D A loop that executes exactly 10 times

8

Which loop is best suited for situations that require a counter?

A for loop

B while loop

C All loops are equally suited

D do-while loop

9

What is the output of the following for loop?

```
for (int i = 1; i <= 3; i++) { cout << i; }
```

A No output

B 1234

C 12

D 123

10

In a for loop, how many times does the initialization expression execute?

A Never

B At the end of each iteration

C Once, at the beginning of the loop

D Every iteration

11

What is the output of the following for loop?
`for (int i = 1; i <= 10; i += 2) { cout << i << " "; }`

A 1 3 5 7 9 11

B 1 3 5 7 9

C 2 4 6 8 10

D 1 2 3 4 5 6 7 8 9 10

12

What is the sum of integers from 1 to 20?

A 190

B 200

C 220

D 210

13

What is the output of the following code?
`int sum = 0; for (int i = 1; i <= 3; i++) { sum = sum + i; } cout << sum;`

A 6

B 3

C 5

D 10

14

What does the break statement do in a loop?

A Restarts the loop from the beginning

B Exits the loop completely

C Pauses the loop execution

D Skips the remainder of the current iteration and continues to the next

15

What is the output of the following code?
`for (int i = 1; i <= 5; i++) { if (i == 3) break; cout << i << " "; }`

A 1 2 3

B 1 2 4 5

C 1 2

D 1 2 3 4 5

16

What does the continue statement do in a loop?

A

Skips the remainder of the current iteration and continues to the next

B

Exits the loop completely

C

Restarts the loop from the beginning

D

Pauses the loop execution

17

What is the output of the following code?

```
for (int i = 1; i <= 5; i++) { if (i == 3) continue; cout << i << " "; }
```

A

1 2 3 4 5

B

1 2

C

1 2 4 5

D

1 2 3

18

What is a sentinel value?

A

A value that increments the counter

B

A value that prints output

C

A value that initializes the counter

D

A value that ends the loop when encountered

19

The do-while loop is a good choice for:

A

Counting from 1 to n

B

Repeating a menu at least once

C

Infinite loops

D

Reading sentinel values

20

What is the output of the following code?

```
int x = 10; while (x > 0) { cout << x; x = x - 2; }
```

A

10 9 8 7 6 5 4 3 2 1

B

10 8 6 4 2

C

10 8 6 4 2 0

D

10 8 6 4



المحاضرة 6

الدوال

Functions

محتويات المحاضرة

1 ما هي الدالة؟

2 لماذا نحتاج الدوال؟

3 تعريف الدوال

4 استدعاء الدوال

5 تمرير المعاملات (Pass by Value / Reference)

6 المتغيرات المحلية والعالمية

7 التحميل الزائد للدوال (Overloading)

8 القيم الافتراضية

9 الدوال العودية (Recursion)

1. ما هي الدالة؟

- الدالة (Function) هي كتلة برمجية صغيرة تؤدي مهمة محددة.
- تُنفذ فقط عندما تُستدعى (called).
- هناك دوال مضمنة (مكتيبة) مثل: `sqrt` ، `pow()` ، ...
- يمكن للمبرمج إنشاء دوال خاصة به (دوال معرفة من قبل المستخدم).

أهمية الدوال

2. لماذا نحتاج الدوال؟

- تقسيم البرنامج إلى مهام صغيرة.
- كتابة البرنامج وتصحيحه جزءاً جزءاً.
- إمكانية عمل عدة مبرمجين على نفس البرنامج.
- تجعل البرنامج أسهل للقراءة والاختبار والتصحيح.
- تقلل حجم الكود (إعادة استخدام الكود).

3. تعريف الدوال

الصيغة العامة

```
return_type function_name ( parameter_type param1 , parameter_type
param2 )
{
    // جسم الدالة
    return value ;
}
```

أمثلة على تعريفات الدوال

```
// دالة لا ترجع قيمة ولا تأخذ معاملات
void displayMessage ( void )
{
    cout << "Hello from the function displayMessage.\n" ;
}

// دالة ترجع مجموع عددين
int sum ( int a , int b )
{
    return a + b ;
}

// دالة لا ترجع قيمة وتطبع المجموع
void printSum ( int a , int b )
{
    int s = a + b ;
    cout << "The sum is: " << s ;
}
```

4. استدعاء الدوال

دالة لا ترجع قيمة

```
displayMessage () ; // استدعاء مباشر
```

دالة ترجع قيمة

```
int result = sum ( 5 , 7 ) ;  
cout << sum ( 3 , 4 ) ; // cout مباشرة في
```

مثال: حساب مساحة المستطيل

```
// تعريف الدالة  
int Area ( int length , int width )  
{  
    return length * width ;  
}  
  
int main ()  
{  
    int x = 5 , y = 6 ;  
    int recArea = Area ( x , y ) ;  
    cout << "The area is: " << recArea ;  
    return 0 ;  
}
```

Enter the length and the width: 5 6

The area is: 30

مثال: دالة غير ناجعة (void)

```
void sum ( int a , int b )
{
    int s = a + b ;
    cout << "The sum is: " << s ;
}

int main ()
{
    sum ( 5 , 7 ) ; // تطبع: The sum is: 12
    sum ( 12 , 33 ) ; // تطبع: The sum is: 45
    return 0 ;
}
```

تمرير المعاملات

5. تمرير المعاملات

التمرير بالقيمة (Pass by Value)

- يتم نسخ قيمة الوسيط (argument) إلى المعامل (parameter) .
- أي تغيير في المعامل لا يؤثر على الوسيط الأصلي.

```
void change ( int x )
{
    x = 100 ; // يغير النسخة فقط
}

int main ()
{
    int num = 5 ;
    change ( num ) ;
    cout << num ; // لا يزال 5
}
```

التمرير بالمرجع (Pass by Reference)

- يتم تمرير **عنوان** المتغير الأصلي.
- أي تغيير في المعامل **يؤثر** على المتغير الأصلي.
- نستخدم & قبل اسم المعامل.

```
void calculate ( int a , int b , int & sum , int & diff )
{
    sum = a + b ;
    diff = a - b ;
}

int main ()
{
    int x = 5 , y = 3 , s , d ;
    calculate ( x , y , s , d ) ;
    cout << "Sum = " << s << ", Difference = " << d ;
    // Sum = 8, Difference = 2
}
```

6. المتغيرات المحلية والعالمية

المتغيرات المحلية (Local Variables)

- تُعرّف داخل دالة.
- لا يمكن الوصول إليها من خارج الدالة.
- تُحذف عند انتهاء الدالة.

```
int sum ( int a , int b )
{
    int r = a + b ; // r محلي
    return r ;
}
```

المتغيرات العالمية (Global Variables)

- تُعرّف خارج جميع الدوال.
- يمكن الوصول إليها من أي دالة في البرنامج.

```
int num = 2 ; // متغير عالمي

void fun ( void )
{
    cout << "num = " << num ; // يمكن الوصول
}
```

تضارب الأسماء

- إذا عُرف متغير محلي بنفس اسم متغير عالمي، يُخفي المتغير المحلي المتغير العالمي داخل الدالة.

المتغيرات الثابتة (Static Variables)

- تحتفظ بقيمتها بين استدعاءات الدالة.
- تُعرّف بـ `static`.

```
void show ( void )
{
    static int num = 5 ;
    cout << "num = " << num << endl ;
    num ++ ;
}

int main ()
{
    show () ; // num = 5
    show () ; // num = 6
    show () ; // num = 7
}
```

7. النموذج الأولي والتحميل الزائد

النموذج الأولي للدالة (Function Prototype)

- إعلان الدالة قبل تعريفها.
- يُكتب قبل main () .
- ينتهي بـ ; (فاصلة منقوطة).

```
int sum ( int , int ) ; // نموذج أولي

int main ()
{
    // هنا sum يمكن استدعاء
}

int sum ( int a , int b ) // تعريف الدالة
{
    return a + b ;
}
```


التحميل الزائد للدوال (Function Overloading)

- يمكن أن يكون لدوال نفس الاسم مع معاملات مختلفة.
- الاختلاف في: عدد المعاملات أو نوع المعاملات.
- يختار المترجم الدالة المناسبة حسب الاستدعاء.

```
int sum ( int a , int b )
{
    return a + b ;
}

int sum ( int a , int b , int c )
{
    return a + b + c ;
}

float sum ( float x , float y )
{
    return x + y ;
}
```

8. القيم الافتراضية

القيم الافتراضية للمعاملات (Default Parameters)

- يمكن إعطاء قيم افتراضية للمعاملات.
- تُستخدم إذا لم يُمرر قيمة عند الاستدعاء.

```
int divide ( int a , int b = 2 )
{
    return a / b ;
}

int main ()
{
    cout << divide ( 12 ) ; // 12/2 = 6
    cout << divide ( 20 , 4 ) ; // 20/4 = 5
}
```

9. الدوال العودية

الدوال العودية (Recursive Functions)

- الدالة التي تستدعي نفسها .
- تُستخدم للمشكلات التي تُعرف بدلالة نفسها.

```
// مثال: حساب المضروب (factorial)
int factorial ( int n )
{
    if ( n <= 1 )
        return 1 ;
    else
        return n * factorial ( n - 1 ) ;
}
```

factorial(5) = 5 × factorial(4)
factorial(4) = 4 × factorial(3)
factorial(3) = 3 × factorial(2)
factorial(2) = 2 × factorial(1)
factorial(1) = 1
→ factorial(5) = 5 × 4 × 3 × 2 × 1 = **120**

النقاط الأساسية

تعريف الدالة: `return_type`
`name(parameters) { body }`

الدوال تقسم البرنامج إلى مهام صغيرة قابلة لإعادة الاستخدام.

التمرير بالقيمة: ينسخ القيمة. التمرير بالمرجع: يمرر العنوان (يستخدم &).

استدعاء الدالة: `name(arguments)`

التحميل الزائد: دوال بنفس الاسم ومعاملات مختلفة.

المتغيرات العالمية: خارج الكل دوال. المحلية: داخل الدالة.

خلاصة سريعة

- الدوال تجعل الكود قابلاً لإعادة الاستخدام وأسهل في الصيانة.
- استخدم `void` إذا كانت الدالة لا ترجع قيمة.
- استخدم `static` للمتغيرات التي تحتفظ بقيمتها بين الاستدعاءات.
- التحميل الزائد يسمح باستخدام نفس اسم الدالة لمهام مختلفة.
- الدوال العودية مفيدة للمشكلات التي تُعرف بدلالة نفسها (مثل المضروب).

1

What is a function in C++?

A

A header file

B

A loop structure

C

A subprogram that performs a specific task

D

A type of variable

2

Which of the following is a library function in C++?

A

main()

B

cout

C

cin

D

pow()

3

What is the correct syntax for a function that does not return a value?

A

float functionName()

B

bool functionName()

C

void functionName()

D

int functionName()

4

What is a function prototype?

A

A function that calls itself

B

A declaration of the function before it is used

C

The body of the function

D

A function with default parameters

5

What is the return type of the following function?

```
int sum(int a, int b) { return a + b; }
```

A

float

B

double

C

int

D

void

6

What is the output of the following code?

```
void display() { cout << "Hello"; }
```

```
int main() { display(); return 0; }
```

A Error

B Hello

C No output

D HelloHello

7

In pass by value, the function receives:

A A reference to the original variable

B A copy of the argument's value

C The original variable itself

D The memory address of the variable

8

In pass by reference, what symbol is used in the parameter list?

A &

B *

C #

D %

9

What is the difference between pass by value and pass by reference?

A Both change the original variable

B There is no difference

C Pass by value changes the original variable; pass by reference does not

D Pass by reference changes the original variable; pass by value does not

10

What is the scope of a local variable?

A The entire program

B The main function only

C All functions in the program

D Only the function where it is declared

11

What is a global variable?

A

A variable declared in the main function

B

A variable declared outside all functions

C

A variable declared inside a function

D

A constant variable

12

What happens when a local variable has the same name as a global variable?

A

The global variable is used

B

Both variables are merged

C

A compiler error occurs

D

The local variable is used (it shadows the global)

13

What is a static local variable?

A

A variable that is deleted after each function call

B

A local variable that retains its value between function calls

C

A variable that is global to all functions

D

A variable that cannot be changed

14

What is function overloading?

A

Having multiple functions with different names

B

A function with default parameters

C

A function that calls itself

D

Having multiple functions with the same name but different parameters

15

What is a default parameter in a function?

A

A parameter that has no data type

B

A parameter that cannot be changed

C

A parameter that is assigned a default value if no argument is passed

D

A parameter that must always be passed

16

What is the output of the following code?

```
int divide(int a, int b=2) { return a/b; }
```

```
int main() { cout << divide(12); return 0; }
```

A 6

B 12

C Error

D 0

17

What is recursion?

A A function that calls itself

B A function that calls another function

C A function with default parameters

D A function with no parameters

18

What is the output of the following code?

```
void fun(int &x) { x = 10; }
```

```
int main() { int a = 5; fun(a); cout << a; return 0; }
```

A 0

B 5

C Error

D 10

19

What is the output of the following code?

```
int square(int n) { return n * n; }
```

```
int main() { cout << square(5); return 0; }
```

A 5

B 50

C 25

D 10

20

Which of the following correctly demonstrates function overloading?

A `int sum(int a); int sum(int a, int b);`

B Both B and C

C `int sum(int a); int sum(int b);`

D `int sum(int a); float sum(float a);`



المحاضرة 7

المصفوفات

Arrays

محتويات المحاضرة

1 ما هي المصفوفة؟

2 المصفوفات أحادية البعد

3 تهيئة المصفوفات

4 المصفوفات النصية (C-Strings)

5 دوال التعامل مع النصوص

6 المصفوفات ثنائية البعد

7 أمثلة تطبيقية

1. ما هي المصفوفة؟

المصفوفة (Array) هي قائمة من العناصر، جميعها من نفس نوع البيانات.

- تُخزن في الذاكرة بشكل متسلسل.
- جميع العناصر لها نفس الاسم.
- يتم تمييز العناصر بواسطة فهرس (index) يبدأ من 0.
- يمكن أن تكون: 1D (أحادية البعد)، 2D (ثنائية البعد)، أو متعددة الأبعاد.

2. المصفوفات أحادية البعد

صيغة التعريف

```
datatype array_name [ number_of_elements ] ;
```

```
;int A [ 5 ]
;float X [ 12 ]
```

تهيئة المصفوفة

```
// الطريقة 1: عند التعريف
; int A [ 5 ] = { 2 , 4 , 6 , 8 , 10 }
// الحجم يُحدد تلقائياً
; int B [ ] = { 1 , 3 , 5 , 7 }
```

```
// الطريقة 2: عنصراً عنصراً
;int hours [ 3 ]
; hours [ 0 ] = 5
; hours [ 1 ] = 4
; hours [ 2 ] = 10
```

```
// الطريقة 3: من المستخدم
;cin >> hours [ 0 ] >> hours [ 1 ] >> hours [ 2 ]
```

ملاحظات مهمة

- الفهرس يبدأ من 0 إلى n-1.
- حجم المصفوفة يجب أن يكون ثابتاً موجباً.
- لا يمكن تعريف مصفوفة بحجم متغير إلا بعد قراءة قيمته.

تابع: التعامل مع المصفوفات

قراءة وكتابة المصفوفة باستخدام for

```
int hours [ 3 ];

// قراءة العناصر
for ( int i = 0 ; i < 3 ; i ++ )
cin >> hours [ i ];

// كتابة العناصر
for ( int i = 0 ; i < 3 ; i ++ )
cout << hours [ i ] << endl ;
```

⚠ أخطاء شائعة

- حجم المصفوفة سالب: `int arr[-1];` ❌
- حجم المصفوفة غير صحيح: `float mean[4.5];` ❌
- حجم المصفوفة متغير بدونتهيئة: `int size; char name[size];` ❌
- تهيئة بعدد عناصر أكثر من الحجم: `int arr[3]={1,2,3,4};` ❌
- الوصول إلى فهرس خارج النطاق: `hours[5]` عندما الحجم 3 ❌ (قد يتسبب في تعطل البرنامج).

القيم غير المهيأة (Garbage Values)

- إذا عُرِّفت مصفوفة محلياً ولم تُهيأ، تحتوي على قيم عشوائية.
- إذا عُرِّفت مصفوفة عاماً أو جزئياً، تُهيأ العناصر غير المحددة بـ 0.

تابع: أمثلة على المصفوفات

مثال 1: الطباعة بترتيب عكسي

```
float a [ 4 ];
cout << "Enter 4 real numbers:\n" ;
for ( int i = 0 ; i < 4 ; i ++ )
cin >> a [ i ];

cout << "Here they are in reverse order:\n" ;
for ( int i = 3 ; i >= 0 ; i -- )
cout << "a[" << i << "] = " << a [ i ] << endl ;
```

مثال 2: جمع مصفوفتين

```
int x [ 3 ] = { 1 , 4 , 7 } ;
int y [ 3 ] = { 5 , 6 , 10 } ;
int z [ 3 ];

for ( int i = 0 ; i < 3 ; i ++ )
z [ i ] = x [ i ] + y [ i ];
```

مثال 3: إيجاد أكبر عنصر

```
int yy [ 6 ];
for ( int i = 0 ; i < 6 ; i ++ )
cin >> yy [ i ];

int largest = yy [ 0 ];
for ( int i = 1 ; i < 6 ; i ++ )
if ( yy [ i ] > largest )
largest = yy [ i ];
cout << "The largest is " << largest ;
```

4. المصفوفات النصية (C-Strings)

ما هي المصفوفة النصية؟

- مصفوفة من الأحرف (char).
- تنتهي بحرف النهاية '\0' (null terminator).
- يجب أن يكون حجمها أكبر بواحد من طول النص المخزن.

```
char Fname [ 4 ] = { 'A' , 'l' , 'i' , '\0' } ;
char Lname [ 6 ] = "Gamal" ; // الحجم 6 يكفي لـ "Gamal" + '\0'
```

⚠ تحذير: نسيان حرف النهاية

```
char name [ 4 ] = { 'C' , 'o' , 'd' , 'e' } ; // لا يوجد '\0'
cout << name ; // قد يطبع أحرفاً عشوائية أو يتعطل البرنامج
```

نسخ المصفوفات النصية

```
char name [ 6 ] = "Gamal" , copy [ 6 ] ;

// طريقة خاطئة
copy = name ; // لا يعمل ❌

// طريقة صحيحة: نسخ حرفاً حرفاً
for ( int i = 0 ; i < 6 ; i ++ )
    copy [ i ] = name [ i ] ; // صحيح ✅
```

5. دوال التعامل مع النصوص

مكتبة cstring

- يجب تضمين `<cstring>`

الدالة	الوصف
<code>strcpy(dest, src)</code>	نسخ src إلى dest
<code>strlen(str)</code>	إرجاع طول النص
<code>strcmp(str1, str2)</code>	مقارنة نصين (0 إذا متساويان)
<code>strchr(str, c)</code>	البحث عن حرف c في النص
<code>toupper(c)</code>	تحويل حرف إلى كبير
<code>tolower(c)</code>	تحويل حرف إلى صغير

مثال: مقارنة النصوص

```
char firstString [ 10 ], secondString [ 10 ];
cout << "Enter a string: " ;
cin >> firstString ;
cout << "Enter another string: " ;
cin >> secondString ;

if ( strcmp ( firstString , secondString ) == 0 )
cout << "You have entered the same string twice.\n" ;
else
cout << "The strings are not the same.\n" ;
```

مثال: التحقق من كلمة المرور

```
char s [ 80 ];  
cout << "Enter password: " ;  
cin >> s ;  
  
if ( strcmp ( s , "password" ) == 0 )  
cout << "Logged on.\nAccess granted" ;  
else  
cout << "Invalid password.\nAccess denied" ;
```


6. المصفوفات ثنائية البعد

صيغة التعريف

```
datatype array_name [ rows ][ columns ] ;
```

```
int A [ 3 ][ 4 ]; // 3 صفوف و 4 أعمدة
float number [ 12 ][ 15 ]
char names [ 5 ][ 20 ]; // 5 أسماء، كل اسم طوله 20 حرفاً
```

تهيئة مصفوفة ثنائية البعد

```
int daytab [ 2 ][ 12 ] = {
{ 31 , 28 , 31 , 30 , 31 , 30 , 31 , 31 , 30 , 31 , 30 , 31 } , //
سنة عادية
{ 31 , 29 , 31 , 30 , 31 , 30 , 31 , 31 , 30 , 31 , 30 , 31 } // سنة
كبيرة
} ;
```

الوصول إلى عناصر المصفوفة

- يتم الوصول باستخدام حلقة متداخلة (nested loop).

```
int Array [ 3 ][ 7 ];

// ملء المصفوفة بالأصفار
for ( int i = 0 ; i < 3 ; i ++ )
for ( int j = 0 ; j < 7 ; j ++ )
Array [ i ][ j ] = 0 ;
```

تابع: أمثلة على المصفوفات ثنائية البعد

مثال 1: قراءة وكتابة مصفوفة 5×3

```
int A [ 3 ][ 5 ];

// قراءة العناصر
for ( int i = 0 ; i < 3 ; i ++ )
for ( int j = 0 ; j < 5 ; j ++ )
cin >> A [ i ][ j ];

// كتابة العناصر
for ( int i = 0 ; i < 3 ; i ++ )
{
for ( int j = 0 ; j < 5 ; j ++ )
cout << A [ i ][ j ] << " " ;
cout << endl ;
}
```

مثال 2: جمع عناصر المصفوفة

```
int A [ 3 ][ 5 ] = { 1 , 2 , 3 , 4 , 5 , 6 , 7 , 8 , 9 , 10 , 11 ,
12 , 13 , 14 , 15 } ;
int sum = 0 ;
for ( int i = 0 ; i < 3 ; i ++ )
for ( int j = 0 ; j < 5 ; j ++ )
sum += A [ i ][ j ];
cout << "The sum is: " << sum ;
```

مثال 3: جمع كل صف على حدة

```
for ( int i = 0 ; i < 3 ; i ++ )  
{  
    int rowSum = 0 ;  
    for ( int j = 0 ; j < 5 ; j ++ )  
        rowSum += A [ i ][ j ] ;  
    cout << "The sum of row " << i << " is: " << rowSum << endl ;  
}
```

تابع: مصفوفة النصوص ثنائية البعد

مصفوفة من النصوص

- كل صف في المصفوفة هو نص (سلسلة أحرف).

```
char array [ 3 ][ 80 ]; // 80 حرفاً 3
// قراءة 3 نصوص
for ( int i = 0 ; i < 3 ; i ++ )
    cin >> array [ i ];

// كتابة النصوص
for ( int i = 0 ; i < 3 ; i ++ )
    cout << array [ i ] << endl ;
```

مثال: قراءة وكتابة مصفوفة 7×3

```
char str [ 3 ][ 7 ];

// قراءة كل حرف على حدة
for ( int i = 0 ; i < 3 ; i ++ )
    for ( int j = 0 ; j < 7 ; j ++ )
        cin >> str [ i ][ j ];

// كتابة كل سطر
for ( int i = 0 ; i < 3 ; i ++ )
{
    for ( int j = 0 ; j < 7 ; j ++ )
        cout << str [ i ][ j ];
    cout << endl ;
}
```

7. عمليات على المصفوفات

جمع مصفوفتين $A[N][M] + B[N][M]$

```
for ( int i = 0 ; i < 3 ; i ++ )
for ( int j = 0 ; j < 2 ; j ++ )
C [ i ][ j ] = A [ i ][ j ] + B [ i ][ j ];
```

ضرب مصفوفة في عدد ثابت $X \times A[N][M]$

```
for ( int i = 0 ; i < 3 ; i ++ )
for ( int j = 0 ; j < 2 ; j ++ )
B [ i ][ j ] = X * A [ i ][ j ];
```

مصفوفة منقولة (Transpose)

- مصفوفة $A[N][M]$ لها مصفوفة منقولة $B[M][N]$.
- $B[j][i] = A[i][j]$

```
for ( int i = 0 ; i < 3 ; i ++ )
for ( int j = 0 ; j < 2 ; j ++ )
B [ j ][ i ] = A [ i ][ j ];
```

```
int n [] = { 19 , 3 , 15 , 7 , 11 , 9 , 13 , 5 , 17 , 1 } ;  
int key , found = 0 ;  
cout << "Enter the value you want to search for: " ;  
cin >> key ;  
  
for ( int i = 0 ; i < 10 ; i ++ )  
{  
    if ( n [ i ] == key )  
    {  
        found = 1 ;  
        cout << "Found at location: " << i ;  
        break ;  
    }  
}  
  
if ( ! found )  
    cout << "Key is not found" ;
```

النقاط الأساسية

الفهرس يبدأ من 0، وحجم المصفوفة يجب أن يكون ثابتاً موجباً. 

المصفوفة هي مجموعة من العناصر من نفس النوع تُخزن بشكل متسلسل. 

دوال: strcpy(), strlen(), strcmp(), strchr() 

المصفوفات النصية تنتهي بـ '\0' ويجب أن يكون حجمها أكبر بواحد. 

البحث الخطي: مقارنة كل عنصر مع القيمة المطلوبة. 

المصفوفات ثنائية البعد: [صفوف] [أعمدة] وتُعالج بحلقات متداخلة. 

خلاصة سريعة

- المصفوفات تسمح بتخزين كميات كبيرة من البيانات بنفس الاسم.
- استخدم for للتعامل مع عناصر المصفوفة.
- المصفوفات النصية تحتاج إلى '\0' في النهاية.
- المصفوفات ثنائية البعد تُستخدم للجداول والبيانات المنظمة.
- انتبه إلى عدم تجاوز حدود المصفوفة لتجنب تعطل البرنامج.

1

What is an array in C++?

A

A list of data objects of the same data type stored sequentially

B

A type of loop

C

A single variable that stores one value

D

A function that returns multiple values

2

What is the index of the first element in a C++ array?

A

1

B

Depends on the array size

C

-1

D

0

3

How do you declare an array of 5 integers in C++?

A

int array(5);

B

array int[5];

C

int[5] array;

D

int array[5];

4

What is the output of the following code?

```
int arr[3] = {5, 4, 10}; for(int i=0; i<3; i++) cout << arr[i] << " ";
```

A

5 4 10

B

0 0 0

C

10 4 5

D

5 4

5

What happens if you initialize an array with fewer elements than its size?

A

Remaining elements get garbage values

B

Both B and C depending on where declared

C

Compiler error

D

Remaining elements get zero (for global arrays)

6

What happens if you initialize an array with more elements than its size?

A

Extra elements are ignored

B

The array size increases automatically

C

Run-time error

D

Compiler error

7

How do you declare an array without specifying its size?

A

```
int arr[] = {1, 2, 3};
```

B

```
int arr[0];
```

C

```
int arr[*];
```

D

```
int arr; int arr[ ];
```

8

What is a string array in C++?

A

An array of floating-point numbers

B

An array of characters ending with '\0'

C

An array of booleans

D

An array of integers

9

What is the null terminator character in C++?

A

```
'\0'
```

B

```
'\n'
```

C

```
'\r'
```

D

```
'\t'
```

10

What does the strcmp() function return when two strings are identical?

A

A positive number

B

1

C

-1

D

0

11

What does the `strlen()` function return?

- ☐ A The length of the string (including `'\0'`)
 - ☐ B The size of the array
 - ☐ C The memory address of the string
 - ☐ D The length of the string (excluding `'\0'`)
-

12

What is the correct syntax to copy a string using `strcpy()`?

- ☐ A `strcpy(source, dest);`
 - ☐ B `strcpy(dest, source);`
 - ☐ C `strcpy(dest = source);`
 - ☐ D `strcpy(dest, source);`
-

13

What is the correct declaration of a 2D array with 3 rows and 4 columns?

- ☐ A `int arr[3,4];`
 - ☐ B `int arr(3)(4);`
 - ☐ C `int arr[3][4];`
 - ☐ D `int arr[4][3];`
-

14

What is a nested for loop?

- ☐ A A for loop that never ends
 - ☐ B A for loop with no body
 - ☐ C A for loop with multiple counters
 - ☐ D A for loop inside another for loop
-

15

In a nested for loop, for each iteration of the outer loop, how many times does the inner loop execute?

- ☐ A Its full number of iterations
 - ☐ B Depends on the outer loop condition
 - ☐ C Zero times
 - ☐ D Once
-

16

What is Bubble Sort?

- A A searching algorithm
- B A type of array declaration
- C A sorting algorithm that compares adjacent elements and swaps them
- D A function to copy arrays

17

What is Linear Search?

- A Searching by dividing the array in half
- B Searching using recursion
- C Searching by comparing each element with the key value
- D Searching with pointers

18

What is the output of the following code?

```
int arr[3][2] = {{1,2},{3,4},{5,6}}; for(int i=0; i<3; i++) for(int j=0; j<2; j++) cout << arr[i][j] << " ";
```

- A 1 2 3 4
- B 1 3 5 2 4 6
- C 1 2 3 4 5 6
- D 6 5 4 3 2 1

19

What does the tolower() function do?

- A Checks if a character is a digit
- B Converts a character to uppercase
- C Checks if a character is a letter
- D Converts a character to lowercase

20

What does the strcat() function do?

- A Compares two strings
- B Copies one string to another
- C Finds the length of a string
- D Concatenates (joins) two strings



المحاضرة 8

المؤشرات

Pointers

محتويات المحاضرة

1 مقدمة

2 أساسيات المؤشرات

3 المصفوفات والمؤشرات

4 تمرير المؤشرات كمعاملات للدوال

1. مقدمة

المؤشرات (Pointers) هي متغيرات مصممة لتخزين عناوين الذاكرة كقيم لها.

- المتغير العادي يحتوي على قيمة محددة (عدد صحيح، عدد عشري، حرف، ...).
- المؤشر يحتوي على عنوان ذاكرة لمتغير آخر يحتوي على قيمة محددة.

أساسيات المؤشرات

2. أساسيات المؤشرات

تعريف المؤشر

```
int count = 5 ;
int * pCount = & count ; // pCount يحتوي على عنوان count
```

- count & → عنوان المتغير count في الذاكرة.
- int* pCount → مؤشر يشير إلى قيمة من نوع int .

مشغل الإشارة (* Dereference Operator)

- pCount* → القيمة الموجودة في العنوان المخزن في المؤشر.

```
cout << * pCount << endl ; // يطبع 5
```

```
#include <iostream>
using namespace std ;

int main ()
{
    int count = 5 ;
    int * pCount = & count ;

    cout << "The value of count is " << count << endl ;
    cout << "The address of count is " << & count << endl ;
    cout << "The address of count is " << pCount << endl ;
    cout << "The value of count is " << * pCount << endl ;
    return 0 ;
}
```

The value of count is 5
The address of count is 00AFF980
The address of count is 00AFF980
The value of count is 5

تابع: أساسيات المؤشرات

الإشارة غير المباشرة (Indirection)

- الوصول إلى قيمة عبر المؤشر يُسمى الإشارة غير المباشرة.

```
count ++ ; // مرجع مباشر
( * pCount ) ++ ; // مرجع غير مباشر
```

نوع المؤشر

- يُصرّح المؤشر بنوع البيانات الذي يشير إليه.
- يجب أن يتطابق نوع المؤشر مع نوع المتغير الذي يشير إليه.

```
int area = 1 ;
double * pArea = & area ; // خطأ! أنواع غير متطابقة
```

تهيئة المؤشر

- يجب تهيئة المؤشرات لتجنب الأخطاء.
- يمكن تهيئة المؤشر بـ `nullptr` أو `0` للإشارة إلى أنه لا يشير إلى أي مكان.
- الإشارة إلى مؤشر غير مهيأ قد تسبب تعطل البرنامج.

```
int * p = nullptr ; // مؤشر لا يشير إلى أي مكان
```

⚠ انتبه!

```
// خطأ: يصرح بمؤشر ومتغير صحيح
int * p1 , pz ; // pz ليس مؤشراً

// صحيح: يصرح بمؤشرين
int * p1 , * pz ;
```

3. المصفوفات والمؤشرات

العلاقة بين المصفوفات والمؤشرات

- اسم المصفوفة بدون قوسين وفهرس يمثل **عنوان البداية** للمصفوفة.
- اسم المصفوفة هو في الأساس **مؤشر**.

```
int list [ 6 ] = { 11 , 12 , 13 , 14 , 15 , 16 } ;
int * p = list ; // p يشير إلى أول عنصر في المصفوفة
```

الوصول إلى عناصر المصفوفة عبر المؤشر

```
// list[0] == *list == *p
// list[1] == *(list + 1) == *(p + 1)
// list[i] == *(list + i) == *(p + i) == p[i]
```

مثال

```
int list [ 6 ] = { 11 , 12 , 13 , 14 , 15 , 16 } ;
int * p = list ;

for ( int i = 0 ; i < 6 ; i ++ )
{
    cout << "address: " << ( list + i ) << " " ;
    cout << "value: " << * ( list + i ) << " " ;
    cout << "value: " << list [ i ] << " " ;
    cout << "value: " << * ( p + i ) << " " ;
    cout << "value: " << p [ i ] << endl ;
}
```

⚠ الفرق بين $list + 1$ و $(list + 1)*$

- $list + 1$ → يضيف 1 إلى **قيمة** العنصر الأول، $(list[0] + 1)$
- $(list + 1)*$ → قيمة العنصر الثاني، $(list[1])$

4. تمرير المؤشرات للدوال

تمرير المؤشر بالقيمة

- يتم نسخ عنوان المؤشر.
- التغيير على p^* يؤثر على القيمة الأصلية.
- التغيير على p نفسه لا يؤثر على المؤشر الأصلي.

تمرير المؤشر بالمرجع

- يتم تمرير مرجع للمؤشر الأصلي.
- التغيير على p نفسه يؤثر على المؤشر الأصلي.

مثال: دالة تأخذ مؤشرين

```
void f ( int * p1 , int * & p2 )
{
    * p1 = 10 ; // OK: القيمة المشار إليها
    p1 = nullptr ; // فقط يغير النسخة المحلية
    * p2 = 20 ; // OK: القيمة المشار إليها
    p2 = nullptr ; // OK: يغير المؤشر الأصلي
}
```

تابع: دوال swap

أربع نسخ من دالة swap

```
// 1. تمرير بالقيمة (لا يعمل)
void swap1 ( int n1 , int n2 )
{
    int temp = n1 ;
    n1 = n2 ;
    n2 = temp ;
}

// 2. تمرير بالمرجع (يعمل)
void swap2 ( int & n1 , int & n2 )
{
    int temp = n1 ;
    n1 = n2 ;
    n2 = temp ;
}

// 3. تمرير مؤشرات بالقيمة (يعمل)
void swap3 ( int * p1 , int * p2 )
{
    int temp = * p1 ;
    * p1 = * p2 ;
    * p2 = temp ;
}

// 4. تمرير مؤشرات بالمرجع (يعمل)
void swap4 ( int * & p1 , int * & p2 )
{
    int * temp = p1 ;
    p1 = p2 ;
    p2 = temp ;
}
```

ملخص الدوال

الدالة	الطريقة	النتيجة
swap1	تمرير بالقيمة	❌ لا يعمل
swap2	تمرير بالمرجع	✅ يعمل
swap3	مؤشرات بالقيمة	✅ يعمل
swap4	مؤشرات بالمرجع	✅ يعمل (يبدل المؤشرات نفسها)

CONST والمؤشرات

const والمؤشرات

مؤشر إلى قيمة ثابتة

- يمكن أن يشير المؤشر إلى قيمة **ثابتة** لا يمكن تغييرها عبر المؤشر.

```
int X = 111 ;
const int * pX = & X ;
* pX = 333 ; // خطأ! لا يمكن تغيير القيمة ❌
```

المصفوفات كمعاملات للدوال

- يمكن استبدال معامل المصفوفة بمعامل مؤشر.

```
// هاتان الدالتان متكافئتان
void m ( int list [], int size )
{ /* ... */ }

void m ( int * list , int size )
{ /* ... */ }
```

النقاط الأساسية

🔗 & → عنوان المتغير، * → قيمة العنوان المخزن.

💡 المؤشرات تخزن عناوين الذاكرة بدلاً من القيم.

☎ تمرير المؤشرات للدوال يسمح بتعديل القيم الأصلية.

📊 اسم المصفوفة هو مؤشر إلى العنصر الأول فيها.

🔒 استخدم `const` مع المؤشرات لمنع التعديل غير المقصود.

↔ `swap3`: مؤشرات بالقيمة، `swap4`: مؤشرات بالمرجع.

🔗 خلاصة سريعة

- المؤشرات تسمح بالتعامل المباشر مع عناوين الذاكرة.
- استخدم * للوصول إلى القيمة، و & للحصول على العنوان.
- المصفوفات والمؤشرات مرتبطان ارتباطاً وثيقاً في C++.
- تمرير المؤشرات للدوال يسمح بتعديل المتغيرات الأصلية.
- احرص على تهيئة المؤشرات لتجنب الأعطال.

1

What is a pointer in C++?

A

A type of array

B

A variable that stores a character

C

A variable that stores the memory address of another variable

D

A function that returns a value

2

What operator is used to get the address of a variable?

A

&

B

*

C

#

D

%

3

What operator is used to dereference a pointer (access the value at the address stored in the pointer)?

A

*

B

&

C

#

D

%

4

Which of the following is the correct way to declare a pointer to an integer?

A

int ptr*;

B

int &ptr

C

*int ptr;

D

int *ptr;

5

What is the output of the following code?

```
int count = 5; int *pCount = &count cout << *pCount;
```

A

5

B

The address of count

C

Error

D

0

6

What is the output of the following code?

```
int count = 5; int *pCount = &count; cout << *pCount;
```

A Error

B The address of count

C 5

D 0

7

What is the correct way to declare two pointers on the same line?

A `int *p1, *p2;`

B `int *p1, p2;`

C `int &p1, &p2`

D `int p1*, p2*;`

8

What does the array name represent without brackets and subscript?

A The size of the array

B The first element of the array

C The last element of the array

D The starting address of the array

9

For an array `int list[6] = {11, 12, 13, 14, 15, 16}`, what does `*(list + 2)` represent?

A 14

B 11

C 12

D 13

10

What is the difference between `*list + 1` and `*(list + 1)`?

A They are the same

B `*list + 1` is `list[1]`; `*(list + 1)` is `list[0] + 1`

C `*list + 1` is `list[0] + 1`; `*(list + 1)` is `list[1]`

D Both cause errors

11

What does it mean to initialize a pointer to 0 or nullptr?

- ☐ A The pointer contains the value 0
- ☐ B The pointer points to address 0
- ☐ C The pointer is invalid
- ☐ D The pointer points to nothing

12

What is the syntax for pass-by-reference with a pointer?

- ☐ A `void func(int &*ptr)`
- ☐ B `void func(int *&ptr)`
- ☐ C `void func(int *ptr)`
- ☐ D `void func(int &ptr)`

13

In the function `void swap3(int *p1, int *p2)`, what is swapped?

- ☐ A The addresses of the pointers
- ☐ B The pointers themselves
- ☐ C The values pointed to by the pointers
- ☐ D Nothing

14

In the function `void swap4(int *&p1, int *&p2)`, what is swapped?

- ☐ A The pointers themselves (their addresses)
- ☐ B Both the values and the pointers
- ☐ C Nothing
- ☐ D The values pointed to by the pointers

15

What does the `const` keyword do when used with a pointer as in `const int *p`?

- ☐ A The pointer cannot be changed
- ☐ B Both the pointer and the value cannot be changed
- ☐ C The value pointed to cannot be changed
- ☐ D The pointer cannot be dereferenced

16

What is the output of the following code?

```
int x = 5; int *p = &x (*p)++; cout << x;
```

A 5

B Error

C 0

D 6

17

What is the output of the following code?

```
int x = 5; int y = 10; int *p = &x p = &y cout << *p;
```

A 5

B 10

C 0

D Error

18

An array parameter in a function can be replaced by a:

A Const parameter

B Pointer parameter

C Value parameter

D Reference parameter

19

What is the output of the following code?

```
int x = 5; int *p = &x cout << &x << " " << p;
```

A Error

B The same address printed twice

C 5 5

D Different addresses

20

What is wrong with the following code?

```
int area = 1; double *pArea = &area
```

A Nothing is wrong

B The pointer is not initialized

C The pointer type does not match the variable type

D The address operator is missing

تنظيم الحاسوب



1 B	6 C	11 A	16 B
2 A	7 D	12 D	17 B
3 C	8 C	13 C	18 D
4 A	9 D	14 C	19 B
5 B	10 A	15 D	20 D



استخدام المتغيرات لتخزين البيانات



1 A	6 C	11 A	16 B
2 B	7 A	12 B	17 C
3 A	8 A	13 C	18 A
4 C	9 D	14 A	19 D
5 D	10 D	15 D	20 C

العمليات الحسابية



1 A	6 B	11 A	16 C
2 D	7 B	12 C	17 D
3 B	8 D	13 D	18 B
4 B	9 D	14 B	19 B
5 D	10 D	15 A	20 C

التحكم في التنفيذ - الاختيار



1 C	6 D	11 D	16 B
2 D	7 D	12 C	17 B
3 C	8 A	13 B	18 B
4 B	9 A	14 D	19 C
5 A	10 C	15 A	20 C

التحكم في التنفيذ - التكرار



1 D	6 A	11 B	16 A
2 D	7 A	12 D	17 C
3 B	8 A	13 A	18 D
4 C	9 D	14 B	19 B
5 A	10 C	15 C	20 B

الدوال



1 C	6 B	11 B	16 A
2 D	7 B	12 D	17 A
3 C	8 A	13 B	18 D
4 B	9 D	14 D	19 C
5 C	10 D	15 C	20 B

المصفوفات



1 A	6 D	11 D	16 C
2 D	7 A	12 D	17 C
3 D	8 B	13 C	18 C
4 A	9 A	14 D	19 D
5 B	10 D	15 A	20 D

المؤشرات



1 C	6 B	11 D	16 D
2 A	7 A	12 C	17 B
3 A	8 D	13 C	18 B
4 D	9 D	14 A	19 B
5 A	10 C	15 C	20 C